

PFAgent: A Tractable and Self-Evolving Power-Flow Agent for Interactive Grid Analysis

Buxin She, *Member, IEEE*, Brian Chen, Luanzheng Guo, *Member, IEEE*, and Fangxing Li, *Fellow, IEEE*

Abstract—Power system simulation workflows remain expert-intensive. Engineers must translate study intents into code or API calls, execute analyses, and interpret outputs. To automate this workflow, this paper presents *PFAgent*, a tractable and self-evolving power-flow agent for interactive grid analysis. *PFAgent* integrates four key capabilities: i) a tractable and interactive architecture for intent parsing, knowledge retrieval, tool execution, and structured reporting; ii) a self-evolution mechanism combining verification-driven refinement and human-in-the-loop feedback; iii) an AI-assisted evaluation and debugging loop that leverages conversational context, generated code, and execution errors for iterative fixing; and iv) an evaluation protocol covering task success, convergence validity, numerical consistency, and explanation quality. Verification on IEEE benchmark systems shows that *PFAgent* can automate case change, analyze voltage and thermal violations, perform N-1 contingency analysis, generate plots and concise summaries, and return reproducible results with transparent execution logs. The proposed framework highlights a shift from conventional simulation tools to interactive, tractable, and self-evolving agents for power system analysis.

Index Terms—Power system agent, large language model, tool-augmented reasoning, interactive grid analysis, power flow

I. INTRODUCTION

POWER-system study workflows are being reshaped by the high penetration of inverter-based resources [1], more frequent extreme-weather events that stress resilience planning [2], fast-growing data-center and electrified loads that alter demand profiles [3], and tighter planning-to-operation feedback loops [4]. Each of these pressures forces analysts to run many more scenario iterations over changing network states and user-defined models [5]. A typical study is procedural as well as numerical. An engineer has to select the correct benchmark or uploaded case, modify loads, setpoints, or topology, rerun power flow, inspect voltages and line quantities, capture plots and tables, and archive outputs for later comparison. Much of this effort is not the computation itself, but the surrounding tool specifications, case management, and reproducible reporting.

Large language models (LLMs) offer a natural-language interface to this workflow, but an engineering study assistant is fundamentally different from a general-purpose chatbot [6], [7]. It has to obey exact simulator APIs, respect the ordering of a power-system workflow, preserve cumulative modifications

across turns, and expose enough structure that its numerical claims can be audited against executable code [8]. In other words, conversational quality is not sufficient; execution correctness, transparency, and reproducibility are first-class requirements. We use the term *power-system agent* to describe systems that convert natural-language study objectives into admissible simulator actions and auditable study outputs under such domain-specific constraints.

Recent literature motivates this framing. Both the opportunities and the limitations of LLMs are discussed in electric-energy systems, emphasizing that domain grounding, tool use, and safety remain essential for practical deployment [9], [10]. Around these perspectives, task-oriented studies have shown that LLMs can assist with fault diagnosis in power grids [11], cybersecurity analysis of smart-grid applications [12], and retrieval-augmented carbon-footprint accounting [13]. These works establish that LLMs are useful in the power and energy domain, but they do not solve the implementation problem of reliable text-to-simulation. Even for simple load-flow code generation, general-purpose models hallucinate technical details and silently alter network specifications [14], and introducing natural-language interfaces raises nontrivial security and reliability concerns that cannot be mitigated by language quality alone [15].

A second line of work moves closer to the workflow requirements of engineering analysis by wrapping LLMs into tool-using agents. Modular prompting and tool feedback can substantially improve LLM performance on previously unseen simulation tools, as shown on Daline [16]. A feedback-driven multi-agent framework in [17] integrates retrieval, reasoning, and environmental-acting modules with an error-feedback loop, and reports strong results on Daline and MATPOWER benchmarks. General LLM-systems building blocks such as retrieval-augmented generation [18] and reasoning-and-acting frameworks [19] underpin these efforts.

Existing works mark clear progress, yet several gaps remain before such agents can serve as practical power system assistants. First, machine-checkable results are uncommon. Models return text or code, but not verifiable structured outputs. Second, existing agents are largely static in their prompt templates and tool configurations; they lack a systematic mechanism that attributes verified failures, updates constraints, and incorporates deployment feedback so that the agent can continuously evolve over successive verification. Third, when generated code fails at runtime, error recovery is typically a shallow regeneration on the same prompt; no existing power-system agent provides a built-in AI-assisted error-fixing capability that uses the full conversational context, the exact failing code, and

B. She is with the Mike Wiegers Department of Electrical and Computer Engineering, Kansas State University, Manhattan, KS 66506 USA.

B. Chen and L. Guo are with Pacific Northwest National Laboratory, Richland, WA 99354 USA.

F. Li is with the Department of Electrical Engineering and Computer Science, The University of Tennessee, Knoxville, TN 37996, USA.

Corresponding author: Buxin She (shebuxin@gmail.com).

the execution error together with repository-level knowledge to produce a repair. Table I summarizes how representative prior works relate to these requirements.

To bridge these gaps, this paper presents *PFAgent*, a tractable and self-evolving power-flow agent for interactive grid analysis. *PFAgent* targets text-to-power-flow simulation and is built on ANDES [20], an open-source Python library for power system analysis. ANDES has a Python API that includes topology and power-flow objects needed for programmatic case editing, N-1 analysis, and post-solve inspection. The agent is organized as a modular pipeline behind a user interface. It parses study intents, retrieves context from the official ANDES manual and case metadata, generates and executes validated Python on an isolated environment, and returns a structured report together with traceable logs and plot results. On top of this pipeline, a self-evolution mechanism turns failures in verification and deployed sessions into updates of the agent's prompts and constraints. An AI-assisted fixing loop uses conversation context, the failing code, and the execution error to repair code with retrieval in real time. The main contributions of this work are as follows.

- A modular agentic architecture that supports interactive and tractable text-to-simulation workflows through intent parsing, knowledge retrieval, tool execution, and structured reporting, together with prompt engineering, fine-tuning, and a dedicated user interface.
- A self-evolution mechanism that integrates self-evaluation, verification, automatic updating, and human-in-the-loop judgment for continuous agent improvement.
- An AI-assisted evaluation and fix loop that sends conversational context, generated code, and execution errors to the debugging pipeline for iterative correction and refinement.
- An evaluation protocol for power system agent performance, including task success, convergence validity, numerical consistency, and explanation quality.

The proposed techniques are implemented in *PFAgent*, which is released as an open-source platform for community-wide evaluation, testing, and further development.¹

The rest of this paper is organized as follows. Section II introduces the design philosophy of the power-system agent and details the four-module technical framework. Section III describes the self-evolution mechanism and the AI-assisted fixing loop. Section IV presents the evaluation protocol, including metrics and an ablation plan. Section V reports the case study on IEEE benchmark systems. Section VI concludes the paper and outlines future work.

II. POWER FLOW AGENTS: DESIGN PHILOSOPHY AND TECHNICAL FRAMEWORK

A. Design Philosophy

A *power-system agent* is defined here as a software system that accepts a natural-language study objective, converts it into a sequence of admissible simulator actions, executes those

actions, and returns a structured, auditable study output. Unlike prompt-engineered question-answering systems, an agent must maintain execution state, enforce domain constraints, and produce outputs whose correctness can be checked independently of the agent itself.

This paper uses power flow tasks as the base implementation scope. Four properties guide the design of *PFAgent*.

- 1) **Tractability.** Every step from user intent to final result must be inspectable. The agent decomposes text-to-simulation into named stages outputs, including parsed intent, retrieved context, generated code, and runtime results. They are observable, editable, and recordable.
- 2) **Interactivity.** The agent supports multi-turn sessions. Earlier case modifications persist unless the user explicitly overrides them, and the user interface provides real-time access to code, execution logs, and error messages.
- 3) **Reproducibility.** Every study output is backed by an executable Python script that runs against the same ANDES case in the same workspace. The result is a machine-parseable JSON structure (`RESULT_JSON`) that encodes all numerical claims.
- 4) **Self-evolution.** The agent is not static. Verified failures are attributed to specific root causes, and the resulting fixes are persisted as updates that take effect in subsequent sessions. Deployment feedback from users enters the same improvement path.

B. Technical Framework

Fig. 1 presents the overall architecture of *PFAgent*, comprising five functional modules. A user interface fronts a four-stage processing pipeline of intent parsing, knowledge retrieval, tool execution, and structured reporting. The user interface includes a setting panel for configuration, a chat panel for natural-language queries and responses, a code-and-execution panel for online inspection, and a plot viewer for generated figures and logs. A delived user interface is presented in Section V.

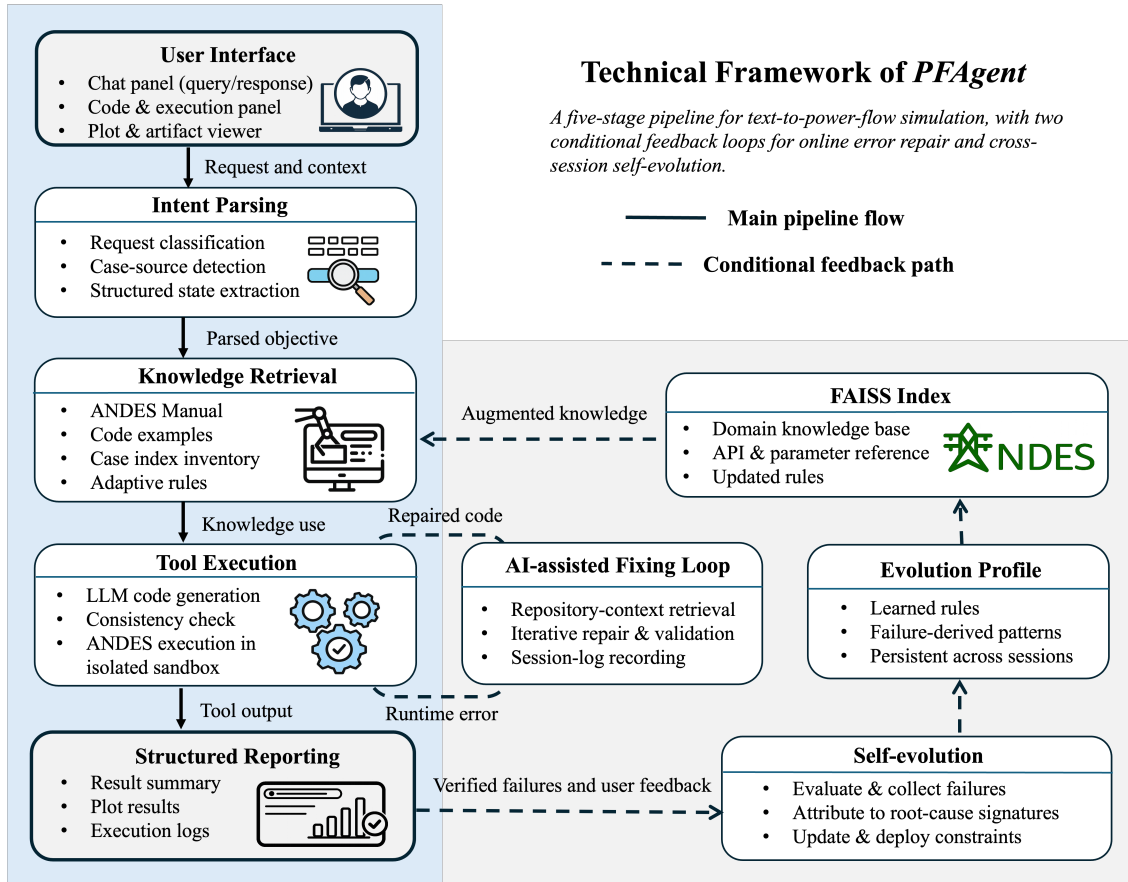
Between modules, there are data flowing along the main pipeline through solid arrows. The user interface passes the request and conversational context to the intent parser, which emits a parsed objective to the knowledge-retrieval stage; the retrieval stage augments the prompt with domain knowledge from a FAISS-indexed ANDES manual, curated code examples, a live case-index inventory, and adaptive rules, then forwards the prompt to the tool-execution stage; execution results are packaged by the structured-reporting stage into a result summary, plot artifacts, and full execution logs.

Two conditional feedback paths (dashed arrows) complement the main pipeline. First, when a runtime error occurs during tool execution, the AI-assisted fixing loop assembles the failing code, the error trace, and repository-level context to produce a repaired script that re-enters the execution stage. Second, verified failures and user feedback collected after reporting are routed to the self-evolution mechanism, which attributes failures to root-cause signatures, updates constraints, and persists the resulting evolution profile. The updated profile feeds back into the FAISS index as adaptive rules, closing a cross-session improvement loop that requires no model retraining.

¹Source code and benchmark scripts are available at the project repository.

TABLE I. *PFAgent* relative to representative prior work on LLMs in power systems.

Ref.	Problem addressed	Executes external simulator	Case-source fidelity	Machine-checkable Output	Self-evolution from verified failures & feedback	Built-in AI-assisted error fixing
[9], [10]	Literature review on LLM use in energy systems	No	No	No	No	No
[11]–[13]	Power-system diagnosis, cybersecurity, and knowledge-retrieval workflows	Not the core task	Limited	Task-specific only	No	No
[14]	Single-turn code generation for OpenDSS load-flow studies	Yes	No	No	No	No
[16]	Simulation-capability improvement on a previously unseen simulator (Daline)	Yes	No	Tool-feedback scoring, no user-facing contract	Static tool feedback only	No
[17]	Cross-tool simulation improvement through retrieval, reasoning, acting, and error feedback	Yes	No	Benchmark success metrics	Error-feedback loop; no failure-driven guardrail update	Regeneration via error signal; no repository repair
PFAgent	Tractable design, self-evolving mechanism, and error recovery	Yes	Yes	Yes, six-aspect scoring criteria	Yes, updates from failure attribution and user feedback	Yes, context-aware AI fixing with iterative local validation

Fig. 1. Technical framework of *PFAgent*. The left column (blue) is the session query pipeline; the right column (gray) contains the feedback loops for error repair and self-evolution.

C. Design of Functional Modules

1) *Intent Parsing*: The first stage classifies the incoming user message and extracts the structured information needed by downstream steps.

- Identify the request type among runnable code, conceptual explanation, and debugging insight. This classification is performed by pattern-based detectors that inspect

the effective user context for markers such as “runnable Python code” or “explain” or “why”.

- Detect the case source. *PFAgent* distinguishes built-in ANDES benchmark cases from user-uploaded files on the exact uploaded filename. Confusing these two paths is one of the most common causes of grounding failure in LLM-generated power-system code.

- Extract the intent markers that specify the study objectives, such as voltage check, load scaling, setpoint adjustment, or N-1 analysis. This object records the case reference, the device parameters, and the accumulated modifications from prior turns. When the codegen gate triggers, the downstream code generation bypasses the LLM entirely and produces a deterministic, template-based Python script that is guaranteed to follow the correct ANDES workflow order.

2) *Knowledge Retrieval*: This stage assembles the context that will form the system prompt for the LLM. Four sources of knowledge are combined.

- The ANDES manual is pre-indexed into a FAISS vector store using sentence-level embeddings. At query time, the retriever selects contiguous page windows from the manual that are most relevant to the current prompt, up to a configurable character budget. This window-based strategy avoids the fragmentation that arises from small, independent chunk retrieval.
- Code examples are loaded from a curated JSON file and injected into the system prompt. These examples demonstrate the expected code style, import order, and output format.
- The case index inventory is built by loading the active ANDES case and rendering the real device-index strings together with their bus connectivity. This eliminates the class of errors in which the LLM guesses a device index instead of using the actual string identifier.
- Adaptive constraints are loaded from the evolution profile. These are prompt-level rules that encode lessons learned from prior verified failures. The set is updated by the self-evolution mechanism described in Section III.

3) *Tool Execution*: This stage converts the prompt into executable Python code and runs it against the ANDES simulator. The process has three phases.

- *Code Generation*: The system prompt and the conversation history are sent to the LLM. The response is normalized to ensure it contains exactly one fenced Python code block, includes necessary imports, and ends with a output statement.
- *Code Inspection*: Before execution, the generated code undergoes two static checks. An AST-level syntax check catches malformed Python. A pattern-based case-loading validator ensures that built-in and uploaded cases are loaded correctly and that device indices are resolved programmatically rather than hardcoded. If either check fails, the error message is fed back to the LLM as a compilation-error message, and the LLM is asked to regenerate. This retry loop runs up to a configurable number of attempts.
- *Code Execution*: Valid code is executed in an isolated environment. The executor captures simulation results and generated plot files. If the code raises an exception during ANDES execution, the error is surfaced to the user together with a “Fix Error with AI” action, which triggers the AI-assisted fixing loop described in Section III.

4) *Structured Reporting*: The final stage packages the execution output into a form that is both user-readable and machine-checkable. The structured output is a JSON object printed by the generated script at the end of execution. Its keys are prompt-specific. For example, a voltage-ranking task include bus index, voltage magnitude, and rank. Its values are the numbers computed by ANDES. This design serves two purposes: it gives the user a concise summary of the study result, and it gives the evaluator a structured output to score. Plot files are captured from the session workspace and rendered in the user interface alongside the conversational response. The full execution log, including the generated code, runtime output, and any error-fix history, is recorded to a session log file. This log ensures that every claim made by the agent can be traced back to a specific script, a specific case state, and a specific ANDES calling.

III. SELF-EVOLUTION AND AI-ASSISTED FIXING MECHANISM

This section introduces two mechanisms that close the loop between agent execution and agent improvement. The self-evolution mechanism operates across sessions. It uses verified failures from benchmark runs and deployment feedback to update the agent’s prompt rules for all future sessions. The AI-assisted fixing loop operates within a single session. When generated code fails at runtime, it invokes a separate repair model with the conversational context, the failing code, the execution error, and repository-level knowledge to produce a fix online.

A. Self-evolution Mechanism

A power-system agent inevitably encounters failures that could not have been anticipated at initial design time. Unseen natural-language phrasings, undocumented edge cases, or format inconsistencies can result in unexpected errors. If these failures are only addressed by re-prompting or manual code patches, the agent remains static and the same class of error recurs in every new session.

PFAgent addresses this with a self-evolution mechanism that operates in two complementary stages, as shown in Fig. 2. In the development stage, the agent is exercised against a programmatically generated benchmark suite before release; in the deployment stage, the same improvement loop continues with real user requests and feedback collected through the user interface. Both stages share a common three-phase processing pipeline, including execute and evaluate, failure attribution, and profile update. In this way, lessons learned in either stage accumulate in the same evolution profile.

1) *Development Stage*: Before the agent is released, a deterministic benchmark suite is constructed by emulating realistic user behavior. Details of the suite are introduced in Section IV. Each scenario in the suite encodes a study session, including a base-case request, a first follow-up with a case modification, and a second follow-up with an additional modification. The scenario design step selects case families, source configurations, and follow-up operation types; the request generation step converts each scenario specification into

natural-language prompts that mimic the phrasing diversity of real users.

The agent is then run on the full suite. Every turn produces generated code, execution outputs, and a structured output in JSON format. The evaluation module scores each turn on six dimensions, including format, grounding, continuity, execution, semantic correctness, and output quality, yielding a per-turn pass/fail label together with a list of specific failure categories. The resulting failure records enter the shared processing pipeline described below.

2) *Deployment Stage*: After release, the same structured logging pipeline records every production session. Each chat turn, code execution event, and user-initiated feedback action is written to a per-session JSON log and appended to a global event stream. These records supplement the development-stage benchmark failures with real-world usage patterns that the benchmark may not cover, such as novel phrasing, unexpected case files, or domain-specific conventions from individual users. A feedback panel integrated into the user interface allows users to flag errors and annotate root causes directly.

3) *Failure Attribution*: Both stages feed failure records into a common attribution step. Each record is matched against a library of failure signatures. A signature is a triple $(\mathcal{P}, \mathcal{E}, \mathcal{I})$ where $\mathcal{P}$ is a set of prompt patterns, $\mathcal{E}$ is a set of execution-error patterns, and $\mathcal{I}$ is a set of human issue patterns. A failure record activates a signature if it matches any pattern in at least one of the three sets.

Each signature is linked to one or more constraint packs. A constraint pack contains three components: (i) prompt guidance sentences that are injected into the system prompt at query time, (ii) pattern overrides that extend the intent parser’s expression vocabulary, and (iii) marker overrides that extend the set of natural-language phrases recognized by the structured code-generation gate.

4) *Profile Update*: The attributed failures are aggregated into an evolution profile. It is a JSON file that records the active constraint packs, the accumulated prompt guidance, the extended pattern and marker overrides, and a root-cause summary with example turns. Profiles from different sources can be merged. The merge operation takes the union of all active packs, deduplicates guidance strings, and accumulates root-cause counts. The merged profile is saved and loaded by the knowledge-retrieval stage at the start of every new session. This ensures that lessons learned from either the development or the deployment stage take effect immediately in all subsequent sessions.

B. AI-Assisted Fixing Loop

The self-evolution mechanism improves the agent over time, but it does not help a user whose code has already failed in the current session. For this case, *PFAgent* provides a built-in AI-assisted fixing loop that operates online within the same conversation.

As shown in Fig. 3, when a generated Python script raises an exception during ANDES runtime execution, the user interface surfaces the error output together with a “Fix Error with AI” action. Clicking this action enqueues a repair request, which is processed as follows.

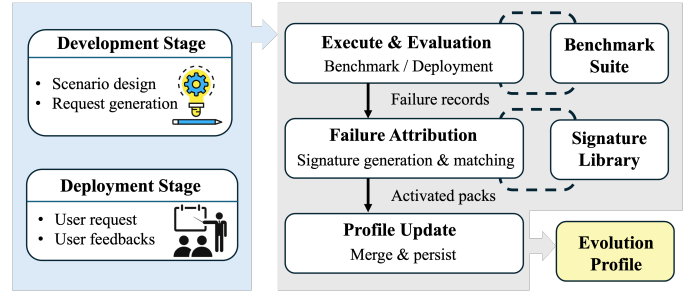


Fig. 2. Self-evolution mechanism.

1) *Context Assembly*: The repair pipeline first assembles a comprehensive fix prompt that combines six sources of context: (i) the original user message that triggered the failing code, (ii) the agent’s response containing the failing code, (iii) the exact Python code block that was executed, (iv) the full output and error messages, (v) the list of files currently available in the session workspace, and (vi) the active ANDES case identifier and continuity state.

In addition, a repository context retriever indexes the PFAgent source tree, including application code, tests, examples, documentation, and verification scripts. This is done by splitting the content into overlapping text chunks. At repair time, it extracts signal terms from the error message and the failing code, and retrieves the top- k repository snippets whose paths or contents best match those terms. This allows the fixer to reference actual ANDES API usage patterns, constraint implementations, and test fixtures, rather than relying on the repair model’s general parametric knowledge.

2) *Repair and Validation*: The assembled prompt is sent to a dedicated repair model, a reasoning model separate from the main chat model. If local validation is enabled, the repaired code is immediately executed in the same session workspace. If execution still produces an error, the new error output replaces the old one in the fix prompt, and the repair model is called again. This iterative cycle continues until the code executes without error or a configurable retry limit is reached.

3) *Termination and Recording*: On success, the validated fix is appended to the conversation history with a note indicating how many repair iterations were needed and whether local validation passed. On failure, the best available fix is presented together with the remaining error output, so the user can diagnose the issue manually. Regardless of outcome, the full repair event is recorded in the session log through the same feedback pipeline used by the self-evolution mechanism. If the repair failure matches one of the known failure signatures, the corresponding constraint pack is queued for activation in the next profile update cycle. This design ensures that the fixing loop and the self-evolution mechanism reinforce each other.

IV. EVALUATION METRICS AND FRAMEWORK

This section defines the metrics used to evaluate *PFAgent* and describes the ablation plan for isolating the contribution of finetuning and RAG techniques.

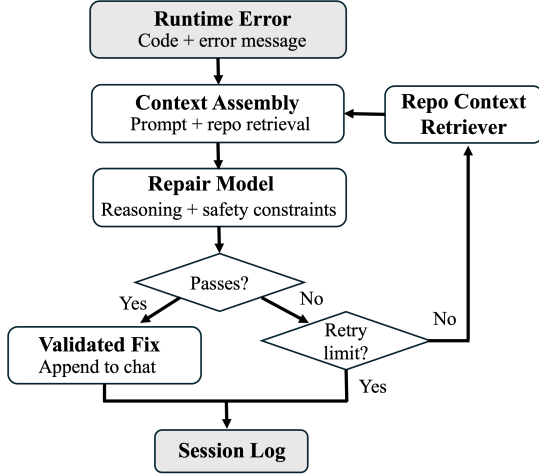


Fig. 3. AI-assisted fixing loop.

A. Metrics

A single task in the *PFAgent* benchmark is a three-turn conversation scenario. The first turn requests a base-case study, and the second and third turns request cumulative case modifications. Each turn is scored individually on six dimensions, and the conversation score is the mean of the three turn scores. A scenario is counted as passed only if all three turns pass, which receives full marks in every dimension. The six dimensions and their point allocations are as follows.

1) *Format (10 points)*: This dimension checks whether the response contains exactly one fenced Python code block. If the response omits code, includes multiple conflicting scripts, or returns prose without executable content when code was requested, the format score is zero. A binary criterion is used because downstream stages cannot proceed without a single, unambiguous code block:

$$S_{\text{fmt}} = \begin{cases} 10, & \text{if exactly one fenced Python block,} \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

2) *Grounding (25 points)*: This dimension checks whether the generated code uses the correct ANDES API calls for the requested task. Each scenario turn specifies a set of code checks, including weighted expression patterns that must appear in the generated script. Examples include the correct `andes.get_case` call for built-in cases, the correct `andes.load` call for uploaded cases, the correct index set for line outages, and the expected output format. A set of forbidden patterns is also checked and any match results in a penalty.

Let $\{(w_j, p_j)\}_{j=1}^J$ be the weighted code checks for a given turn, where w_j is the weight and p_j is a binary indicator for whether pattern j was found in the code. The grounding score is:

$$S_{\text{gnd}} = 25 \times \frac{\sum_{j=1}^J w_j \cdot p_j}{\sum_{j=1}^J w_j}. \quad (2)$$

3) *Continuity (15 points)*: Multi-turn studies require that earlier modifications persist in later turns unless the user

explicitly overrides them. Each follow-up turn specifies carry-forward checks. The weighted patterns must appear in the generated code to confirm that prior edits such as a load scaling or a PV setpoint change from a previous turn are still present. The scoring formula is identical to (2), scaled to 15 points:

$$S_{\text{cont}} = 15 \times \frac{\sum_{j=1}^{J'} w_j \cdot p_j}{\sum_{j=1}^{J'} w_j}. \quad (3)$$

4) *Execution (20 points)*: This dimension checks whether the generated code runs to completion in the ANDES runtime without raising a Python exception. The code is executed in an isolated sandbox with a code prefix that redirects matplotlib output to files. The criterion is binary:

$$S_{\text{exec}} = \begin{cases} 20, & \text{if exit code} = 0, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

Note that a zero execution score does not necessarily imply wrong reasoning. It may result from a missing import, a type mismatch in a device index, or an ANDES API call that is valid but incompatible with the current version.

5) *Numerical Consistency (25 points)*: This dimension compares the output against an expected result computed independently by the verification module. The verification module loads the same ANDES case, applies the same cumulative operations using ANDES API calls, runs power flow, and extracts the expected key-value pairs.

Let $\{k_1, \dots, k_K\}$ be the keys specified by the verification module for a given turn. For each key k_i , the expected value v_i^* and the agent-produced value v_i are compared. Numeric values are compared with an absolute tolerance of 10^{-4} ; string and integer values are compared exactly. The semantic score is:

$$S_{\text{sem}} = 25 \times \frac{|\{i : |v_i - v_i^*| \leq 10^{-4}\}|}{K}. \quad (5)$$

6) *Output Quality (5 points)*: For turns that request a plot, the evaluation checks whether (a) the expected plot filename appears in the `RESULT_JSON`, and (b) the corresponding image file was created in the output directory:

$$S_{\text{art}} = \begin{cases} 5, & \text{if plot file is produced and correctly named,} \\ 0, & \text{otherwise (or if no plot was requested).} \end{cases} \quad (6)$$

7) *Aggregation*: The total turn score is:

$$S_{\text{turn}} = S_{\text{fmt}} + S_{\text{gnd}} + S_{\text{cont}} + S_{\text{exec}} + S_{\text{sem}} + S_{\text{art}}, \quad (7)$$

with a maximum of 100. A turn passes if and only if it receives full marks in all six dimensions. The conversation score is the mean of the three turn scores, and a scenario passes only if all three turns pass.

B. Ablation Plan

The evaluation framework supports four agent modes that form a natural ablation sequence.

1) **Base OpenAI**. The base LLM receives the user prompt without retrieval, without fine-tuning, and without ANDES-specific constraints. This mode serves as

the lower-bound baseline, measuring how far a general-purpose LLM can go when given only the task description.

- 2) **Fine-tuned.** A fine-tuned variant of the same base model, trained on a corpus that pools canonical ANDES code examples, LLM-generated prompt/code pairs, hand-curated hard cases, and oracle-verified single- and multi-turn conversations. The pooled samples are deduplicated and split 90/10 into train/validation JSONL files. This mode isolates the effect of domain-specific training data without retrieval.
- 3) **RAG.** The base LLM augmented with the full knowledge-retrieval pipeline and the structured codegen gate. The retrieval layer indexes the official ANDES manual into a FAISS vector store with window-level passages, and injects a curated library of canonical code examples as few-shot context. This mode isolates the effect of retrieval and prompt engineering without fine-tuning.
- 4) **Fine-tuned + RAG.** The fine-tuned model combined with the full retrieval pipeline. This is the recommended production mode and the upper bound of the ablation.

V. CASE STUDY

This section introduces the delivered *PFAgent* and evaluates it on a deterministic benchmark suite covering representative power-flow study tasks.

A. Delivered Platform

The four-stage pipeline described above is delivered through a Streamlit-based web interface. Fig. 4 shows the two main panels of the interface. The configuration panel provides sidebar controls for API key entry and prompt customization, agent-mode selection, with validation toggles, AI-assisted fixing options, conversation history management, case-file upload, and session persistence. The chat panel hosts the interactive study session, comprising a conversational code-generation area, an error diagnostics view, an AI-assisted code-repair interface, a user-feedback and logging module that feeds the self-evolution mechanism, a root-cause attribution display, and an inline result-visualization pane for plots and output.

B. Study Setup

1) *Benchmark Suite:* The evaluation uses a deterministic scenario suite generated for demonstration. Two suite sizes are reported: an initial 100-scenario suite used during early development, and an expanded 164-scenario suite that adds more challenging tasks introduced after the first round of self-evolution. Each scenario consists of three turns, including a baseline request (Turn 1), a first follow-up with a case modification (Turn 2), and a second follow-up with an additional modification (Turn 3).

2) *Case Families and Sources:* Four IEEE benchmark case families are used: IEEE 14, IEEE 39, Kundur, and PJM 5. Each family appears in two source configurations, including built-in cases loaded from the ANDES package via `andes.get_case()` and user-uploaded cases `andes.load()`.

3) *Task Types:* The follow-up modifications cover a cross-section of power-flow analysis tasks, including: load addition before setup, uniform scaling after setup, slack-bus and PV-bus voltage setpoint adjustment, targeted load modification at a specific bus, targeted generation change at a specific bus, line outage by bus pair, N-1 contingency analysis over a candidate line list, voltage and line-angle ranking with threshold filtering, and voltage-profile plot generation.

4) *Agent Modes:* Four modes introduced in Section IV-B are evaluated and compared on the benchmark suite under the same settings.

5) *Self-Evolution Comparison:* To measure the effect of the self-evolution mechanism, the Fine-tuned + RAG mode is evaluated on the 164-scenario suite both before and after the evolution profile is updated from the failure-attribution pass. The pre-evolution run uses the initial prompt template, while the post-evolution run uses the updated constraint packs derived from the first round of verified failures.

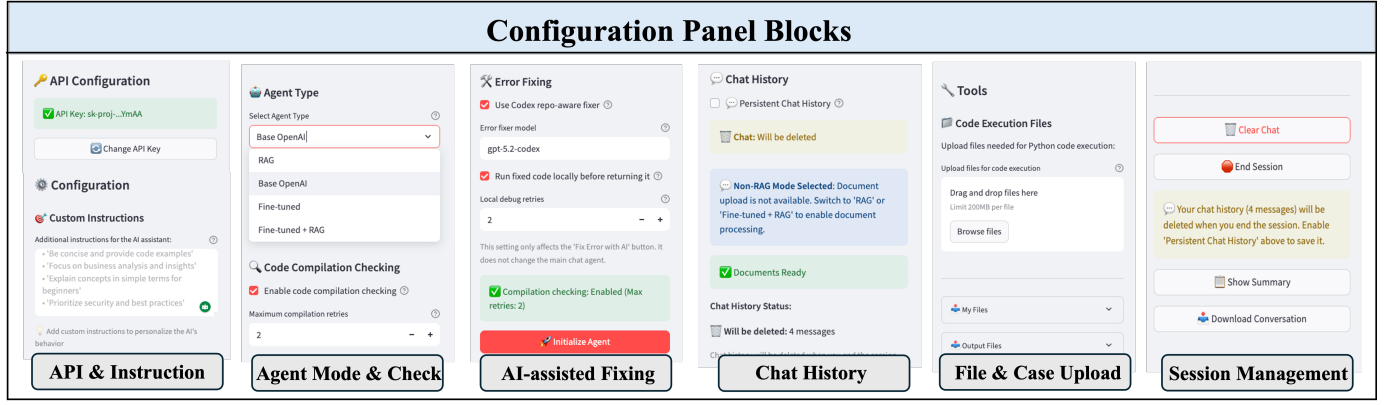
C. Results

1) *100-Scenario Benchmark:* Fig. 5(a) visualizes the metrics result for the 100-scenario benchmark. The retrieval-backed modes both achieve a 100% scenario pass rate and a perfect average conversation score of 100.0, meaning every turn in every scenario received full marks on all six evaluation dimensions. The Fine-tuned mode without retrieval reaches 58% scenario pass rate with an average score of 86.44. The Base OpenAI mode, which lacks both retrieval and domain tuning, does not pass any scenario and averages 46.35.

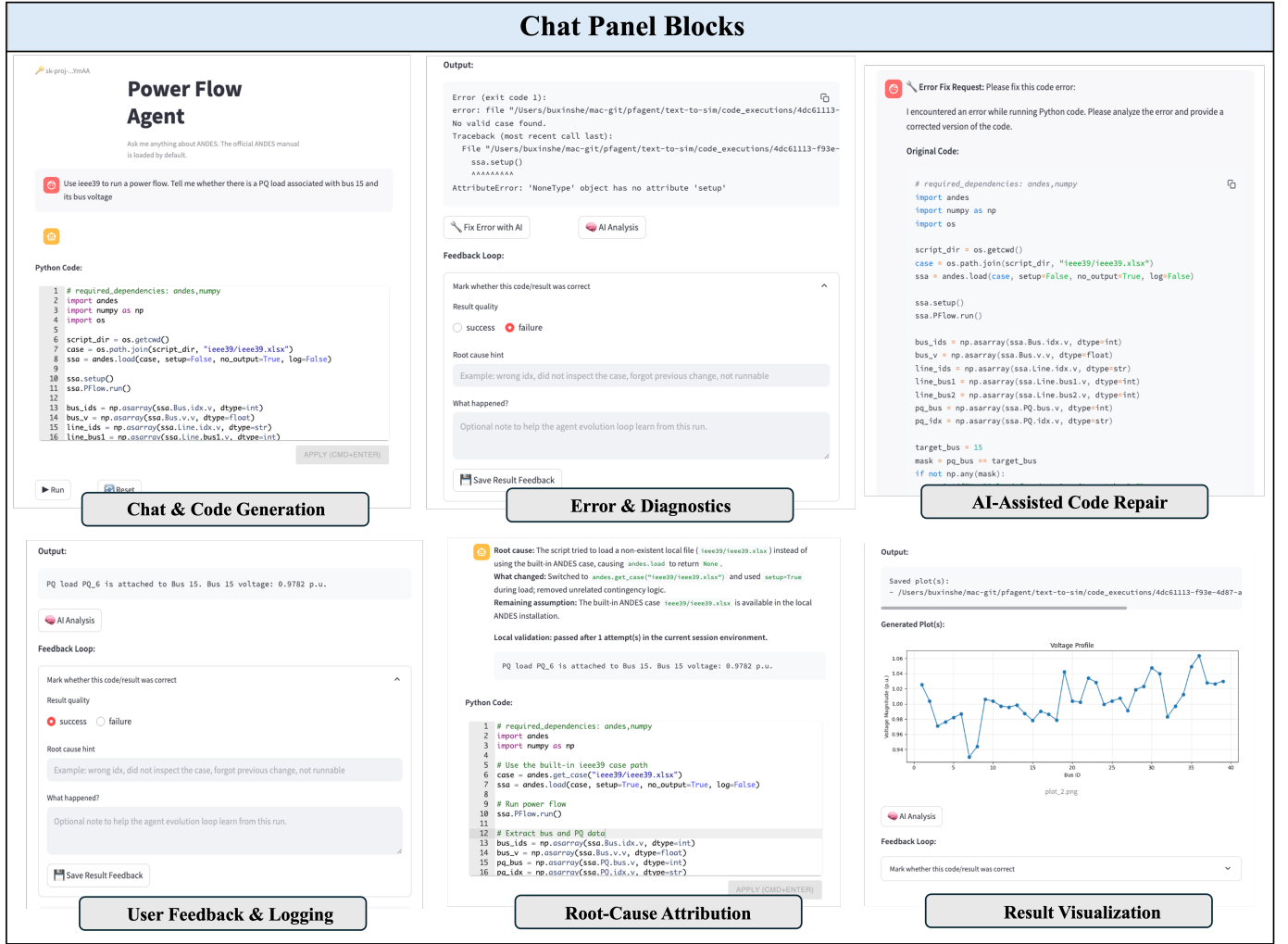
2) *Turn-Level Analysis:* Fig. 5(b) shows the turn-level breakdown. Two patterns are visible. First, the Fine-tuned mode degrades across turns: Turn 1 pass rate is 85%, Turn 2 drops to 68%, and Turn 3 drops further to 66%. This indicates that the fine-tuned model struggles to preserve cumulative modifications in later turns without the explicit continuity support provided by the retrieval pipeline. Second, the retrieval-backed modes show no degradation at all. Their per-turn pass rate is 100% for all three turns.

3) *Dimension-Level Analysis:* Fig. 5(c) shows the average score as a percentage of the maximum for each evaluation dimension. The Base OpenAI mode scores well on format (87%) and reasonably on grounding (84%) and continuity (86%), meaning the LLM usually produces a single code block with roughly correct API structure. However, it fails completely on execution (0%) and semantics (0%): the generated code does not run, and consequently no JSON output is produced. The Fine-tuned mode improves execution to 77% and semantics to 76%, but these remain the weakest dimensions. The retrieval-backed modes achieve 100% on all six dimensions.

4) *Failure Analysis:* Fig. 6 shows the failure-category distribution for the Base OpenAI and Fine-tuned modes. RAG and Fine-tuned + RAG have zero failures, so they are omitted. For Base OpenAI, all 300 turns fail on execution and semantics. The grounding failures account for 240 out of 300 turns, indicating widespread API misuse. For Fine-tuned, the dominant failure categories shift to semantics (81) and execution (69). It means the model produces code that is closer



(a)



(b)

Fig. 4. User interface of PFAgent. (a) Configuration panels; (b) Chat panels.

to correct but still contains errors that prevent convergence or produce wrong numerical results.

5) *Family-Level Analysis*: For the retrieval-backed modes, all eight case-family variants achieve 100% scenario pass rate. For the Fine-tuned mode, performance varies by family and source. Fig. 7 shows that built-in IEEE 14 reaches 85.7% while PJM 5 (both sources) reaches only 37.5%. The uploaded vari-

ants consistently score lower than their built-in counterparts, confirming that case-source fidelity is a genuine challenge when retrieval and explicit loading rules are absent.

6) *Self-Evolution Effect*: Fig. 8 report the before/after comparison on the expanded 164-scenario suite.

Before the evolution update, the Fine-tuned + RAG mode achieves a 60.98% scenario pass rate with an average score

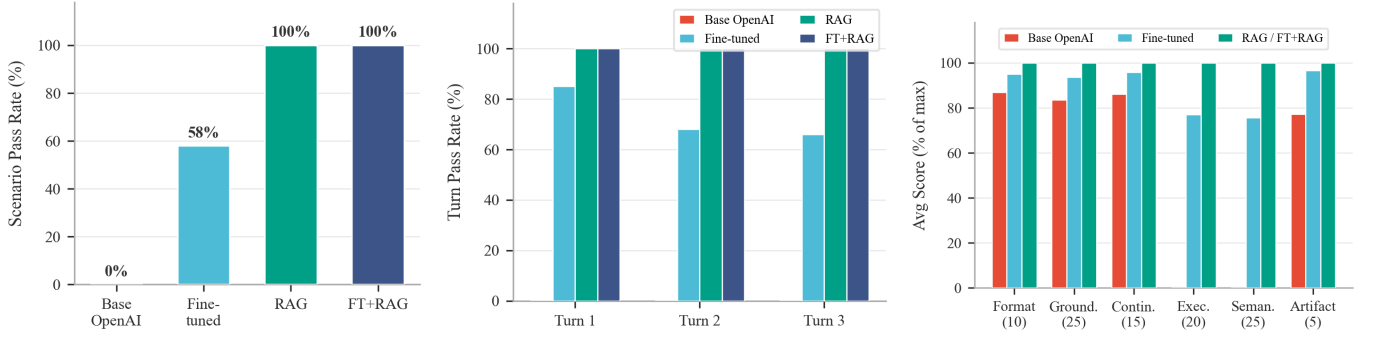


Fig. 5. 100-scenario benchmark results: (a) Scenario pass rate by mode. (b) Turn-level pass rate; (c) Dimension-level performance comparison.

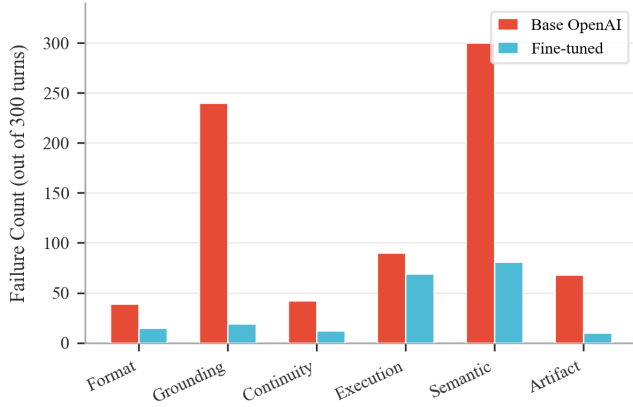


Fig. 6. Failure-category distribution for Base OpenAI and Fine-tuned modes on the 100-scenario benchmark.

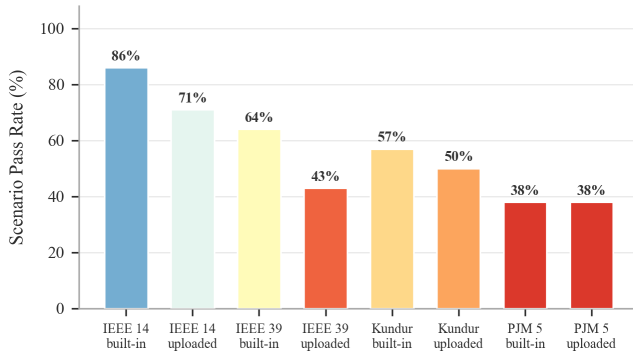


Fig. 7. Family-level scenario pass rate for the Fine-tuned mode.

of 99.14. Turns 1 and 2 pass at 100%, but Turn 3 drops to 60.98%. Failure analysis reveals that nearly all Turn 3 failures are grounding failures concentrated in contingency tasks, where the agent uses an API form that is semantically correct but does not match the grounding contract.

After updating the evaluation profile with new constraint packs for line-outage API alignment, islanding inspection, and corridor phrasing, the same mode is rerun on the same 164-scenario suite. The scenario pass rate recovers to 100.0%, and

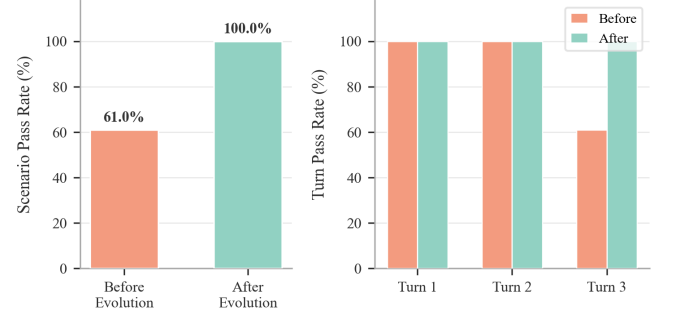


Fig. 8. Self-evolution before/after comparison on the 164-scenario benchmark (Fine-tuned + RAG): (a) Scenario pass rate; (b) Turn-level analysis.

all 492 turns pass with a perfect score. No model retraining is involved. The improvement comes entirely from the updated constraint packs injected into the system prompt.

D. Discussion

1) Retrieval as the Dominant Factor: The most striking result is that retrieval-backed modes achieve 100% pass rate on both the 100-scenario and the post-evolution 164-scenario suites, while fine-tuning alone reaches only 58% (100 scenarios) or 34.76% (164 scenarios). Moreover, the RAG mode ties with Fine-tuned + RAG (fine-tuned model + retrieval) at 100% in both suites. This indicates that, for the current task scope and benchmark design, the retrieval and prompt-engineering pipeline is sufficient to bridge the gap between a general-purpose LLM and a reliable power-flow agent. Fine-tuning does not degrade performance when combined with retrieval, but it does not provide measurable additional benefit either.

One interpretation is that the fine-tuning dataset, while curated and validated, is not yet large enough to internalize the full range of ANDES API patterns, case-loading rules, and device-index conventions that the retrieval pipeline provides dynamically at query time. Producing substantially more fine-tuning data, particularly multi-turn sequences with cumulative modifications, may be needed before fine-tuning can match retrieval on these tasks. This observation is consistent with the broader finding in [17] that retrieval and tool feedback are

more impactful than model scale alone for simulation code generation.

2) *Multi-Turn Continuity as a Differentiator*: The turn-level analysis reveals that Base OpenAI and Fine-tuned both degrade across turns, with the largest drop between Turn 1 and Turn 2. In contrast, retrieval-backed modes maintain 100% at every turn. This is because the retrieval pipeline explicitly injects i) the active ANDES case identifier and continuity state, ii) the full case idx inventory, and iii) the conversation compaction summary into every prompt. This provides the LLM with the ground-truth state it needs to generate code that preserves prior modifications. Without this explicit context, the LLM must reconstruct the cumulative state from the conversation history alone, which becomes increasingly unreliable as the number of modifications grows.

3) *Self-Evolution without Retraining*: The 164-scenario before/after comparison demonstrates that the self-evolution mechanism can recover from a 60.98% pass rate to 100.0% without any model retraining. The root cause of the pre-evolution failures was a mismatch between the ANDES runtime's line-outage API and the grounding patterns embedded in the agent's prompt templates. The failure-attribution step correctly identified this as the dominant signature, including `corridor_outage_language` and `line_outage_api_guardrail`, and the resulting constraint packs updated the system prompt to use the correct API form.

This result supports a practical development workflow for power-system agents, which include six key steps.

- Expand the benchmark with harder and more open-ended tasks.
- Run the agent and collect failures.
- Use failure clustering to locate the dominant bottleneck.
- Determine whether the bottleneck is model capability, agent workflow, or verifier mismatch.
- Update the evolution profile accordingly.
- Rerun the same suite to measure recovery.

This cycle can be repeated indefinitely, with the same base LLM, until the agent reaches the desired level of reliability and coverage.

VI. CONCLUSION

This paper presented *PFAgent*, a tractable and self-evolving power-flow agent for interactive grid analysis on the ANDES platform. By integrating intent parsing, knowledge retrieval, tool execution, structured reporting, self-evolution, and an AI-assisted fixing loop, the proposed framework enables text-to-simulation workflows. The case studies show that retrieval-augmented grounding is the key factor for reliable multi-turn performance, while the self-evolution mechanism can effectively recover from dominant failure modes through prompt- and rule-level updates without retraining the base model. The results suggest that reliable power-system agents should be built as execution-grounded and continuously improvable systems, rather than being treated as standalone language models.

The current study is limited to steady-state power-flow analysis and the benchmark scope constructed in this work.

Future work will extend the framework to optimal power flow, dynamic and stability analysis, and cross-simulator settings. Overall,

ACKNOWLEDGMENT

This work was funded by the startup fund of the Department of Electrical and Computer Engineering, Kansas State University.

REFERENCES

- [1] B. She, F. Li, H. Cui, J. Wang, Q. Zhang, and R. Bo, "Virtual inertia scheduling (VIS) for real-time economic dispatch of ibr-penetrated power systems," *IEEE Transactions on Sustainable Energy*, vol. 15, no. 2, pp. 938–951, 2023.
- [2] M. Panteli, P. Mancarella, D. N. Trakas, E. Kyriakides, and N. D. Hatziaargyriou, "Power systems resilience assessment: Hardening and smart operational enhancement strategies," *Proceedings of the IEEE*, vol. 105, no. 7, pp. 1202–1213, 2017.
- [3] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey, "Recalibrating global data center energy-use estimates," *Science*, vol. 367, no. 6481, pp. 984–986, 2020.
- [4] B. Kroposki, B. Johnson, Y. Zhang, V. Gevorgian, P. Denholm, B.-M. Hodge, and B. Hannegan, "Achieving a 100% renewable grid: Operating electric power systems with extremely high levels of variable renewable energy," *IEEE Power and Energy Magazine*, vol. 15, no. 2, pp. 61–73, 2017.
- [5] B. She, R. R. Hossain, S. Kundu, M. Elizondo, and V. Adetola, "Hybrid symbolic-numerical modeling and parametric stability analysis of DC-AC power systems," *IEEE Open Access Journal of Power and Energy*, 2026.
- [6] Y. Cheng, W. Liu, Y. Xue, J. Huang, J. Zhao, and F. Wen, "Leveraging large language model based agent for automated electricity market modelling and simulation," *Journal of Modern Power Systems and Clean Energy*, 2025.
- [7] H. Zhao, Y. Cheng, D. Xiang, X. Zhou, J. Zhao, X. Cai, and Z. Dong, "Large language model-based power dispatch agent: Framework, application and challenges," *International Journal of Electrical Power & Energy Systems*, vol. 175, p. 111653, 2026.
- [8] H. Jin, K. Kim, and J. Kwon, "Gridmind: Llm-powered agents for power system analysis and operations," in *Proceedings of the SC'25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 560–568.
- [9] S. Majumder, L. Dong, F. Doudi, Y. Cai, C. Tian, D. Kalathil, K. Ding, A. A. Thatte, N. Li, and L. Xie, "Exploring the capabilities and limitations of large language models in the electric energy sector," *Joule*, vol. 8, no. 6, pp. 1544–1549, 2024.
- [10] C. Huang, S. Li, R. Liu, H. Wang, and Y. Chen, "Large foundation models for power systems," in *2024 IEEE Power & Energy Society General Meeting (PESGM)*. IEEE, 2024, pp. 1–5.
- [11] J. Liu and A. Rahman, "Fault diagnosis in power grids with large language model," *arXiv preprint arXiv:2407.08836*, 2024.
- [12] A. Zabolli, S. L. Choi, T.-J. Song, and J. Hong, "ChatGPT and other large language models for cybersecurity of smart grid applications," in *2024 IEEE Power & Energy Society General Meeting (PESGM)*. IEEE, 2024, pp. 1–5.
- [13] H. Wang, M. Zhang, Z. Chen, N. Shang, S. Yao, F. Wen, and J. Zhao, "Carbon footprint accounting driven by large language models and retrieval-augmented generation," *arXiv preprint arXiv:2408.09713*, 2024.
- [14] R. S. Bonadia, F. C. Trindade, W. Freitas, and B. Venkatesh, "On the potential of chatgpt to generate distribution systems for load flow studies using OpenDSS," *IEEE Transactions on Power Systems*, vol. 38, no. 6, pp. 5965–5968, 2023.
- [15] J. Ruan, G. Liang, H. Zhao, G. Liu, X. Sun, J. Qiu, Z. Xu, F. Wen, and Z. Y. Dong, "Applying large language models to power systems: Potential security threats," *arXiv preprint arXiv:2311.13361*, 2024.
- [16] M. Jia, Z. Cui, and G. Hug, "Enabling large language models to perform power system simulations with previously unseen tools: A case of Daline," *arXiv preprint arXiv:2406.17215*, 2024.
- [17] —, "Enhancing LLMs for power system simulations: A feedback-driven multi-agent framework," *IEEE Transactions on Smart Grid*, 2025.

- [18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [20] H. Cui, F. Li, and K. Tomsovic, “Hybrid symbolic-numeric framework for power system modeling and analysis,” *IEEE Transactions on Power Systems*, vol. 36, no. 2, pp. 1373–1384, 2021.